



LAB EXERCISES  
**Distributed Algorithms (IN4150)**

Distributed Systems (DS)  
Department Software Technology  
Faculty EEMCS

---

**Assignment 3**

CAUSAL-ORDER BROADCAST

---

2024-2025

## Assignment

In this lab assignment, you will implement a causal-order Byzantine broadcast algorithm. The algorithm is designed to work in a complete network where up to  $f$  of the  $N$  processes can be Byzantine. Please ensure that the solution adheres to the following requirements:

1. The algorithm is implemented using the Python template and the IPv8 networking library (available on Brightspace).
2. The implementation runs across multiple Docker containers.
3. The network structure and the node behaviour are adjustable (e.g., network size and messages sent).
4. Each node incorporates random delays before sending a message in order to emulate network conditions (e.g., through traffic control).
5. Each node logs events in a **human-readable** fashion (e.g., to a log file or terminal).
6. The code is submitted to the GitLab repository.

## Causal Order Broadcast

We aim to implement a distributed algorithm that enforces the Reliable Causal-Order broadcast abstraction (rco) on a (virtual) fully connected network. RCO requires the following properties:

**RCO-Validity** If a correct process  $p$  broadcasts a message  $m$ , then every correct process eventually RCO-delivers  $m$ .

**RCO-No duplication** No correct process RCO-delivers a message  $m$  more than once.

**RCO-Integrity** If a correct process RCO-delivers a message  $m$  with sender  $p_i$ , then  $m$  was previously broadcast by  $p_i$ .

**RCO-Agreement** If some correct process RCO-delivers a message  $m$ , then every correct process eventually RCO-delivers  $m$ .

**RCO-Causal Order** No correct process  $p_i$  RCO-delivers a message  $m_2$  unless it has already delivered every message  $m_1$  such that  $m_1 \rightarrow m_2$ .

## Part A: Waiting Causal Order Broadcast

**Figure 1** details the pseudocode of the reliable causal-order broadcast protocol you have to implement. This particular RCO variant is called *waiting causal broadcast* [1]. It makes use of vector clocks (noted VC) and of a reliable broadcast protocol (cf. assignment 2).

---

**Algorithm 3.14** Waiting Causal Broadcast

---

**Implements:**

ReliableCausalOrder(rco).

**Uses:**

ReliableBroadcast (rb).

**upon event**  $\langle \text{init} \rangle$  **do**for all  $p_i \in \Pi$  **do**  $\text{VC}[\text{rank}(p_i)] := 0$ ; $\text{pending} := \emptyset$ ;**procedure** deliver-pending **is****while exists**  $(s_x, [\text{DATA}, \text{VC}_x, x]) \in \text{pending}$  **such that** $\forall_{p_j} : \text{VC}[\text{rank}(p_j)] \geq \text{VC}_x[\text{rank}(p_j)]$  **do** $\text{pending} := \text{pending} \setminus (s_x, [\text{DATA}, \text{VC}_x, x])$ ;**trigger**  $\langle \text{rcoDeliver} \mid s_x, x \rangle$ ; $\text{VC}[\text{rank}(s_x)] := \text{VC}[\text{rank}(s_x)] + 1$ ;**upon event**  $\langle \text{rcoBroadcast} \mid m \rangle$  **do****trigger**  $\langle \text{rcoDeliver} \mid \text{self}, m \rangle$ ;**trigger**  $\langle \text{rbBroadcast} \mid [\text{DATA}, \text{VC}, m] \rangle$ ; $\text{VC}[\text{rank}(\text{self})] := \text{VC}[\text{rank}(\text{self})] + 1$ ;**upon event**  $\langle \text{rbDeliver} \mid p_i, [\text{DATA}, \text{VC}_m, m] \rangle$  **do****if**  $p_i \neq \text{self}$  **then** $\text{pending} := \text{pending} \cup (p_i, [\text{DATA}, \text{VC}_m, m])$ ;

deliver-pending;

Figure 1: Reliable Causal-Order Broadcast

## Part B: Testing & Reporting

Do not forget to evaluate your code.

For the sign-off, **be ready to demonstrate** that:

- Two causally related messages are always delivered in the right order by all correct processes.
- Total order is not enforced, i.e., all messages are not always delivered in the same order by all correct processes.
- Ways in which faulty nodes can attempt to disrupt the protocol.

### Report

In this report (**PDF** format), you will present your implementation and benchmarking of a distributed algorithm with minimal text. Focus on presenting clear metrics and plots that demonstrate your algorithm's performance under different conditions. Include at least the following.

#### 1. Title page

- List all group members and the algorithm implemented.
- Briefly describe the purpose of the algorithm (max. 4 sentences).

#### 2. Experimental Setup

Summarize the key parameters you used in a **table** (e.g., number of nodes, frequency of message drops, failure rate).

#### 3. Results

For each metric, provide a brief description (few sentences) followed by the relevant plots. Ensure plots are clear and labeled, with axis labels and a title. Use consistent units across plots (e.g., seconds for time, messages/sec for throughput). Minimum set of plots to provide:

- Latency and message complexity for a RCO-broadcast operation.
- Discussion of Byzantine deviations identified and their impact.

#### 4. Conclusion

Summarize the key findings from your plots. For example: "Our implementation scales well up to 10 nodes, but latency significantly increases with larger networks."

## Tips

- Avoid lengthy descriptions or text blocks; use bullet points or tables where possible.
- Focus on clear, high-quality visuals that allow for quick interpretation.
- Use plot types appropriate for the metric (e.g., line, box, bin).
- Use a sufficient number of points for your plots, e.g., 10 for a line.

## References

- [1] C. Cachin, R. Guerraoui, and L. E. T. Rodrigues, *Introduction to Reliable and Secure Distributed Programming*. 2011.